Date:27.02.2026	Principal Component Analysis(PCA)
Ex. No: 4	

AIM:

To implement the Support Vector Channel **(SVM)** classification algorithm on the Breast Cancer *Dataset* and evaluate its performance **with and without Principal Component Analysis (PCA)** for dimensionality reduction.

ALGORITHM:

Algorithm SVM_PCA(X_train, X_test, y_train, y_test, n_components) return success or failure

X_train : (x1,x2,…,xn training features)

X_test: (diagnosis test feature)

y_train: (x1,x2,…,xn testing features)

y_test: (diagnosis test feature)

n_components: no.of. principal components

Import necessary libraries

Checking for null values using df.isnull().sum()

Encode the diagnosis column using Label Encoder()

Prepare mean centered matrix for the columns

x_mean = (x1+x2+…xn) / len(df)

A_c = (x1[i] - x_mean, x2[i]-x_mean , …, xn[i]-x_mean)

Calculate the covariance matrix

Cov (A_c) = (1/ n-1) *(A_c)

Calculate Eigen Values

Eig_values = det (Cov(A_c) - lambda (I)) = 0

Calculate Eigen Vectors

Eig_vectors = det(Cov(A_c) - Eig_values (I)) =0

Normalize the Eigen Vectors

Norm_Eig_vectors = $(1/\sqrt{\sum Eig_vectors^2})$

Calculate the principal components

pca_components= Norm_Eig_vectors * (A_c)

Pass pca_components to the svm

svm.fit(pca_components)

Weights and bias are calculate using : $w^T + b = 0$

return accuracy, classification_report

EVALUATION METRICS

1. Accuracy

$$\boldsymbol{Accuracy = (TP + TN)/(TP + TN + FP + FN)}$$

2. Precision

$$\boldsymbol{Precision = TP/(TP + FP)}$$

3. Recall (Sensitivity)

$$\boldsymbol{Recall = TP/(TP + FN)}$$

4. F1-Score

$$\boldsymbol{F1Score = 2 * (Precision * Recall)/(Precision + Recall)}$$

CODE:

 # Load data

```
import pandas as pd

df = pd.read_csv(r'breast-cancer.csv')

df.head()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430

5 rows × 32 columns

```
df.shape
```

(569, 32)

```
df.columns
```

```
Index(['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
       'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave points_worst',
       'symmetry_worst', 'fractal_dimension_worst'],
      dtype='object')
```

df.isnull().sum()

```
id                         0
diagnosis                  0
radius_mean                0
texture_mean               0
perimeter_mean             0
area_mean                  0
smoothness_mean            0
compactness_mean           0
concavity_mean             0
concave points_mean        0
symmetry_mean              0
fractal_dimension_mean     0
radius_se                  0
texture_se                 0
perimeter_se               0
area_se                    0
smoothness_se              0
compactness_se             0
concavity_se               0
concave points_se          0
symmetry_se                0
fractal_dimension_se       0
radius_worst               0
texture_worst              0
```

df.dtypes

df.isnull().sum().sum()

output: 0

df.shape

(569, 32)

```
df=df.drop('id',axis=1)

from sklearn.preprocessing import LabelEncoder
encoder=LabelEncoder()
df['diagnosis']=encoder.fit_transform(df['diagnosis'])

X=df.drop('diagnosis',axis=1)
y=df['diagnosis']

X.shape

from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
X_scaled=scaler.fit_transform(X)
```

WITHOUT PCA:

```
from sklearn.svm import SVC
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

svm=SVC()
svm.fit(X_train,y_train)

y_pred=svm.predict(X_test)

acc= accuracy_score(y_test,y_pred)
classification= classification_report(y_test,y_pred)

print(f"Accuracy at {i} no of components is {acc}")
print("Classification report:")
print(classification)
```

```
Accuracy at 1 no of components is 0.9473684210526315
Classification report:
              precision    recall  f1-score   support

           0       0.92      1.00      0.96        71
           1       1.00      0.86      0.92        43

    accuracy                           0.95       114
   macro avg       0.96      0.93      0.94       114
weighted avg       0.95      0.95      0.95       114
```

WITH PCA

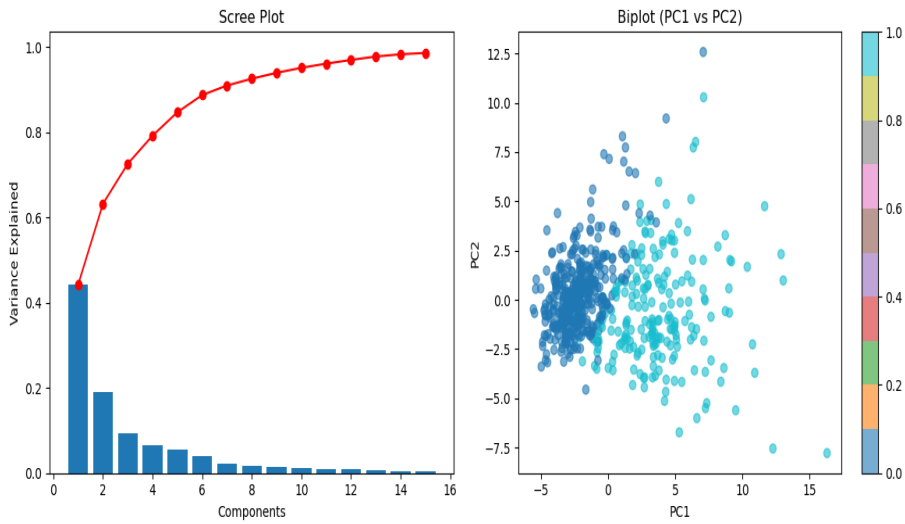
```
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report


for i in [25,20,15,10,5,2,1]:
    pca= PCA(n_components=i)
    X_pca=pca.fit_transform(X_scaled)
    X_train, X_test, y_train, y_test = train_test_split(X_pca, y, test_size=0.2, random_state=42)

    svm=SVC()
    svm.fit(X_train,y_train)

    y_pred=svm.predict(X_test)

    acc= accuracy_score(y_test,y_pred)
    classification= classification_report(y_test,y_pred)

    print(f"Accuracy at {i} no of components is {acc}")
    print("Classification report:")
    print(classification)

    print('*********************************************')
```

OUTPUTS:

```
Accuracy at 25 no of components is 0.9736842105263158
Classification report:
              precision    recall  f1-score   support

           0       0.97      0.99      0.98        71
           1       0.98      0.95      0.96        43

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114

*********************************************
```

```
**********************************************
Accuracy at 15 no of components is 0.9824561403508771
Classification report:
              precision    recall  f1-score   support

           0       0.97      1.00      0.99        71
           1       1.00      0.95      0.98        43

    accuracy                           0.98       114
   macro avg       0.99      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114

**********************************************
Accuracy at 10 no of components is 0.9736842105263158
Classification report:
              precision    recall  f1-score   support

           0       0.97      0.99      0.98        71
           1       0.98      0.95      0.96        43

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114

**********************************************
**********************************************
Accuracy at 5 no of components is 0.9649122807017544
Classification report:
              precision    recall  f1-score   support

           0       0.97      0.97      0.97        71
           1       0.95      0.95      0.95        43

    accuracy                           0.96       114
   macro avg       0.96      0.96      0.96       114
weighted avg       0.96      0.96      0.96       114

**********************************************
Accuracy at 2 no of components is 0.9649122807017544
Classification report:
              precision    recall  f1-score   support

           0       0.95      1.00      0.97        71
           1       1.00      0.91      0.95        43

    accuracy                           0.96       114
   macro avg       0.97      0.95      0.96       114
weighted avg       0.97      0.96      0.96       114

**********************************************
```

```
*********************************************
Accuracy at 1 no of components is 0.9298245614035088
Classification report:
              precision    recall  f1-score   support

           0       0.91      0.99      0.95        71
           1       0.97      0.84      0.90        43

    accuracy                           0.93       114
   macro avg       0.94      0.91      0.92       114
weighted avg       0.93      0.93      0.93       114

*********************************************
```



INFERENCE :

Thus the accuracy of the model increased with the no of principal components and it helps the svm model to learn the best parameters and make good generalization.